Secure Software Development Lifecycle (SSDLC)

Transparency Hub Network Version: 1.0 **Effective Date:** March 5, 2026 **Last Updated:** March 5, 2026 **Owner:** Engineering Team

---

1. Overview

This document defines the Secure Software Development Lifecycle (SSDLC) practices adopted by Transparency Hub Network. It ensures that security is integrated into every phase of software development — from requirements gathering through deployment and maintenance.

Transparency Hub Network handles sensitive membership data, financial transactions, and personal information. These practices protect our users, their organizations, and our platform.

---

2. Scope

This policy applies to:

- All application code (frontend, backend, mobile)
- Infrastructure and deployment configurations
- Third-party integrations and APIs
- Internal tools and scripts
- All contributors (employees, contractors, open-source contributors)

Repositories covered:

- `Transparency-Hub/Staticwebsite-App` — Admin dashboard (Next.js)
- `Transparency-Hub/StaticWebsite` — Public website (Next.js)
- `Transparency-Hub/AzureFunction` — Backend API (Python / Azure Functions)
- `Transparency-Hub/docs` — Documentation (Mintlify)
- Any future repositories under the `Transparency-Hub` GitHub organization

---

3. Development Phases

3.1 Requirements & Design

Security Requirements Gathering:

- All new features must include a security impact assessment before development begins
- Features handling PII (personally identifiable information), financial data, or authentication must undergo a threat modeling exercise
- Acceptance criteria must include security requirements (e.g., "input must be validated", "endpoint must require authentication")

Threat Modeling:

- Use STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) for threat identification
- Document threats and mitigations in the feature's GitHub issue or design document
- High-risk features (payments, OAuth, data exports) require sign-off from a senior engineer

Data Classification:

Classification	Examples	Handling
Public	Marketing content, documentation	No restrictions
Internal	Architecture diagrams, meeting notes	Access-controlled
Confidential	Member PII, email addresses, phone numbers	Encrypted at rest and in transit
Restricted	Payment credentials, OAuth tokens, API keys	Encrypted, access-logged, rotated regularly

3.2 Secure Coding Standards

General Principles:

- Follow the principle of least privilege in all code
- Never trust user input — validate and sanitize all inputs on both client and server
- Use parameterized queries for all database operations (SQLAlchemy ORM)
- Avoid hardcoding secrets, credentials, or API keys in source code
- Use environment variables or secret management services for all sensitive configuration

Language-Specific Standards:

Python (Backend — Azure Functions):

- Use type hints for all function signatures
- Use SQLAlchemy ORM with parameterized queries — never raw SQL with string interpolation
- Validate request payloads using schema validation before processing
- Use `logging` module — never `print()` with sensitive data
- Pin all dependencies in `requirements.txt` with exact versions
- Use virtual environments (`.venv`) for local development

TypeScript/JavaScript (Frontend — Next.js):

- Enable TypeScript strict mode
- Use Zod schemas for runtime validation of API responses and form inputs
- Sanitize any HTML content before rendering (especially user-generated content like meeting minutes)
- Use `next/headers` and server-side API routes for sensitive operations
- Never expose API keys or secrets in client-side code
- Use `httpOnly` and `secure` flags on all cookies

Environment Variables:

- All secrets stored in `local.settings.json` (backend) and `.env.local` (frontend) — both gitignored
- Production secrets managed via Azure Key Vault or Azure Functions Application Settings
- Never commit `.env` , `local.settings.json` , or any file containing secrets

3.3 Source Control & Branching

Branch Strategy:

- `main` — Production-ready code. Protected branch.
- Feature branches — `feature/<description>` or `feat/<description>`
- Bug fix branches — `fix/<description>`
- All changes to `main` must go through a Pull Request

Branch Protection Rules (GitHub):

- Require at least 1 approving review before merge
- Require status checks to pass (lint, build, tests)
- Dismiss stale reviews when new commits are pushed
- No direct pushes to `main`
- No force pushes to `main`

Commit Hygiene:

- Use conventional commits: `feat:`, `fix:`, `chore:`, `docs:`, `security:`
- Never include secrets, credentials, or PII in commit messages
- Sign commits with GPG keys where possible

3.4 Code Reviews

All code changes must be reviewed before merging. No exceptions.

Reviewer Checklist:

- [ ] Input validation — Are all user inputs validated and sanitized?
- [ ] Authentication — Are all endpoints properly authenticated?
- [ ] Authorization — Does the code enforce proper access control (role-based)?
- [ ] Data exposure — Does the response expose only necessary fields?
- [ ] Error handling — Are errors handled gracefully without leaking internal details?
- [ ] SQL injection — Are all queries parameterized?
- [ ] XSS — Is user-generated content properly escaped/sanitized?
- [ ] Secrets — Are there any hardcoded secrets, tokens, or credentials?
- [ ] Dependencies — Are new dependencies from trusted sources with no known vulnerabilities?
- [ ] Logging — Does logging avoid capturing sensitive data (passwords, tokens, PII)?
- [ ] Tests — Are security-relevant changes covered by tests?

Review Requirements:

- All PRs require at least 1 reviewer
- Security-sensitive changes (auth, payments, OAuth, data access) require 2 reviewers
- Reviewer must not be the PR author
- Reviews must be completed within 2 business days

3.5 Secrets Management

Policy: No secrets in source code. Ever.

Where Secrets Live:

Environment	Storage	Access
Local development	`local.settings.json` / `.env.local` (gitignored)	Developer machine only
Staging	Azure Functions App Settings	Azure RBAC
Production	Azure Key Vault + App Settings	Azure RBAC, access-logged

Secret Types and Rotation Schedule:

Secret	Rotation	Method

Database credentials	Every 90 days	Azure Key Vault rotation
OAuth client secrets (Google, Zoom, Teams)	Every 180 days or on compromise	Provider dashboard + Key Vault update
Attendee.dev API key	Every 180 days or on compromise	Attendee dashboard + App Settings
Webhook signing secrets	Every 180 days or on compromise	Regenerate + update both sides
JWT signing keys	Every 90 days	Key Vault rotation
Payment gateway keys (Stripe/Paystack)	Every 180 days or on compromise	Provider dashboard + Key Vault
Gemini API key	On compromise	Google Cloud Console

Secret Scanning:

- GitHub Secret Scanning enabled on all repositories
- Pre-commit hooks scan for potential secrets before allowing commits
- Any detected secret is treated as compromised and rotated immediately

Incident Response for Leaked Secrets:

1. Revoke/rotate the compromised secret immediately
2. Audit access logs for unauthorized usage
3. Notify affected parties if user data may be impacted
4. Document the incident and update procedures to prevent recurrence

3.6 Dependency Management

Policy: All dependencies must be vetted, version-pinned, and regularly scanned.

Dependency Scanning:

- GitHub Dependabot enabled on all repositories for automated vulnerability alerts
- `npm audit` (frontend) and `pip audit` (backend) run as part of CI pipeline
- Critical and high-severity vulnerabilities must be patched within 7 days
- Medium-severity vulnerabilities must be patched within 30 days

Adding New Dependencies: Before adding a new package:

1. Verify the package is actively maintained (recent commits, responsive maintainers)
2. Check for known vulnerabilities on Snyk or npm audit
3. Review the package's license compatibility
4. Prefer packages with >1,000 weekly downloads and established reputation
5. Document the reason for the dependency in the PR description

Lock Files:

- `package-lock.json` (Node.js) and `requirements.txt` with pinned versions (Python) must be committed
- Never use `*` or `>=` version ranges in production dependencies

3.7 Testing

Security Testing Requirements:

Test Type	When	Tool/Method
Unit tests	Every PR	pytest (backend), Jest (frontend)
Input validation tests	Every PR touching API endpoints	pytest with invalid/malicious payloads
Authentication tests	Every PR touching auth	Verify unauthenticated requests are rejected
Authorization tests	Every PR touching access control	Verify role-based access enforcement
Webhook signature tests	On webhook changes	HMAC-SHA256 verification tests
Integration tests	Before release	End-to-end API testing
Dependency scan	Every PR (CI)	`npm audit` / `pip audit` / Dependabot

Test Coverage:

- Minimum 70% code coverage for new code
- Security-critical paths (auth, payments, data access) must have >90% coverage
- All bug fixes must include a regression test

3.8 CI/CD Pipeline

Continuous Integration (GitHub Actions):

Every PR triggers:

1. **Lint** — ESLint (frontend), flake8/ruff (backend)
2. **Build** — Verify the project compiles without errors
3. **Test** — Run full test suite
4. **Dependency audit** — `npm audit` / `pip audit`
5. **Secret scan** — Check for accidentally committed secrets

Continuous Deployment:

- Staging deploys automatically on merge to `main` (after all checks pass)
- Production deploys require manual approval
- Rollback procedure documented and tested quarterly

Pipeline Security:

- CI/CD secrets stored in GitHub Encrypted Secrets
- Deployment credentials use short-lived tokens where possible
- Pipeline logs are reviewed for sensitive data leaks

3.9 Infrastructure Security

Azure Configuration:

- All data encrypted at rest (Azure Storage Service Encryption, PostgreSQL TDE)
- All data encrypted in transit (TLS 1.2+ enforced)
- Network access restricted via Azure Virtual Network where applicable
- Azure Functions use HTTPS-only with minimum TLS 1.2
- Static Web Apps enforce HTTPS with automatic certificate management

Database Security:

- PostgreSQL access restricted to Azure Functions via firewall rules
- Connection strings stored in Azure Key Vault
- Database backups encrypted and retained per retention policy
- No direct database access from developer machines in production

Blob Storage Security:

- Private access only (no public blob access)
- Shared Access Signatures (SAS) with short expiry for temporary access
- Storage account firewall restricts access to Azure Functions

3.10 API Security

Authentication:

- All API endpoints require JWT authentication (except public health checks)
- Tokens validated on every request with signature verification
- Token expiry enforced (1-hour access tokens, 7-day refresh tokens)

Authorization:

- Role-based access control (RBAC) enforced at the controller level
- Members can only access data within their chapter
- Admin actions require admin role verification
- Association-level actions require association admin role

Rate Limiting:

- API rate limiting enforced (100 requests/minute per user)
- Authentication endpoints have stricter limits (10 requests/minute)
- Rate limit responses include `Retry-After` header

Input Validation:

- All request bodies validated against schemas before processing
- File uploads restricted by type and size
- URL parameters sanitized to prevent injection attacks

CORS:

- CORS configured to allow only known frontend origins
- Credentials mode restricted to same-origin requests
- Preflight cache set to reasonable TTL

3.11 Logging & Monitoring

What We Log:

- All authentication events (login, logout, failed attempts)
- Authorization failures (access denied)
- API errors (4xx, 5xx)
- Data modification events (create, update, delete on sensitive entities)
- Payment events (success, failure, refund)
- Webhook events (received, verified, failed verification)

What We Never Log:

- Passwords or password hashes
- Full credit card numbers
- OAuth tokens or API keys
- Personal data beyond what's needed for debugging (minimize PII in logs)

Monitoring:

- Azure Application Insights for performance and error monitoring
- Alerts configured for: authentication failures spike, error rate increase, unusual API patterns
- Log retention: 90 days in Application Insights, 1 year in cold storage

3.12 Incident Response

Severity Levels:

Level	Description	Response Time	Examples
P0 — Critical	Active exploitation, data breach	Immediate (< 1 hour)	SQL injection exploited, credentials leaked publicly
P1 — High	Vulnerability with high impact, no active exploitation	< 4 hours	Auth bypass found, unpatched critical CVE
P2 — Medium	Vulnerability with limited impact	< 24 hours	XSS in non-critical page, medium CVE
P3 — Low	Minor security improvement	Next sprint	Informational finding, best practice gap

Incident Response Steps:

1. **Detect** — Alert received or vulnerability reported
2. **Triage** — Assess severity and impact
3. **Contain** — Isolate affected systems, revoke compromised credentials
4. **Remediate** — Fix the vulnerability, deploy patch
5. **Recover** — Verify fix, restore normal operations
6. **Review** — Post-incident review, update procedures, document lessons learned

Responsible Disclosure:

- Security issues can be reported to: security@transparencyhubnetwork.ai
- We commit to acknowledging reports within 48 hours
- We commit to providing a fix timeline within 7 days

---

4. Third-Party Integrations

All third-party services must meet these requirements:

Requirement	Details
Data encryption	TLS 1.2+ in transit, encryption at rest
SOC 2 or equivalent	Compliance certification required for services handling PII
Data processing agreement	Required for services processing member data

API security	Authentication required, rate limiting in place
Incident notification	Provider must notify us of breaches within 72 hours

Current Third-Party Services:

Service	Purpose	Data Shared
Attendee.dev	Meeting bot recording	Meeting URLs, bot scheduling metadata
Google Gemini	AI meeting minutes	Meeting transcripts (anonymized where possible)
Stripe	Payment processing	Payment amounts, customer email
Paystack	Payment processing (Africa)	Payment amounts, customer email
Google OAuth	Meeting provider auth	OAuth tokens (managed by Google)
Zoom (via Attendee.dev)	Meeting provider auth	OAuth tokens (managed by Attendee.dev)
Microsoft OAuth	Meeting provider auth	OAuth tokens (managed by Microsoft)

---

5. Compliance & Privacy

- All data handling complies with the Nigeria Data Protection Regulation (NDPR)
- GDPR principles followed for international users
- Privacy Policy published at transparencyhubnetwork.ai/privacy
- Terms of Service published at transparencyhubnetwork.ai/terms-of-service
- Data Subject Access Requests (DSAR) handled within 30 days
- Right to deletion honored — member data can be purged on request

---

6. Training & Awareness

- All developers must review this SSDLC document upon onboarding
- Security best practices refresher conducted quarterly
- OWASP Top 10 awareness required for all developers
- Incident response drill conducted annually

---

7. Document Control

Version	Date	Author	Changes
1.0	2026-03-05	Engineering Team	Initial version

This document is reviewed and updated quarterly, or immediately following a significant security incident.

For questions about this policy, contact: support@transparencyhubnetwork.ai